

Ray Tracing + ML Pipeline — Model Reference

Sionna 0.19 Calibration → Sionna 2 DEM Final Results

June 2026

Scene: Nottingham, UK

Frequency: 915.95 MHz

TX: Ofcom 2018

Section 1 — Pipeline Overview

The pipeline operates in two stages. Stage 1 (Calibration) runs in *sionna019_differentiable_rt_fixed.ipynb* using Sionna 0.19 differentiable RT. It calibrates physical and statistical models against 120 drive-test receivers. Stage 2 (Simulation) runs in *sionna2_915mhz_dem_simulation.ipynb* using Sionna 2.0 with a Digital Elevation Model. It loads calibrated parameters from Stage 1 and produces the final RSSI maps.

Pipeline Flow Diagram

[Drive-test RSSI]	→	[Sionna 0.19 RT]	→	[Cell 10b]	→	scalar_offset.json				
			→	[Cell 11b]	→	calibrated_materials.json	→	[Sionna 2 DEM Cell 4A]	→	RSSI_sim
			→	[Cell 16]	→	material_mlp.npz	■			
			→	[Cell 15]	→	residual_mlp.npz	→	POST-PROCESS	→	RSSI_final
			→	[Cell 14]	→	cnn_residual.h5	→	POST-PROCESS	→	RSSI_final

Section 2 — Model Descriptions

2.1 Cell 10b — Global Scalar Offset

Property	Value
Type	Single-parameter calibration (1 tf.Variable)
Input	rss_i_sim (all 120 calib_receivers), rss_i_meas (drive-test)
Method	Minimise SMAPE loss over 200 steps, Adam LR=0.1. Best-RMSE checkpoint.
Output	scalar_offset_915mhz.json → {"scaling_factor_db": +5.8024}
Sionna 2	Cell 4A reads scalar_offset_915mhz.json, adds offset to all simulated RSSI
Result	RSSI_sim_corrected = RSSI_sim + 5.80 dB
RMSE	7.90 dB (N=120, all calib receivers)
Portable	No — must be re-run per scene/frequency

2.2 Cell 11b — Differentiable RT Material Calibration (NVLabs)

Property	Value
Type	Per-material ϵ_r , σ , S calibration via tf.Variable (102 variables)
Input	173 receivers (0–1.2 km, mat_calib_receivers), drive-test RSSI
Method	MAE loss on RSSI, Tikhonov regularisation $\lambda=0.001$, 500 steps, Adam cosine LR 0.05→0.001. No gradient p
Active mat.	14/102 (concrete, brick, glass, metal, asphalt — only materials visible to rays)
Output	cell11b_calibrated_materials.json + cell11b_checkpoint.json
Sionna 2	Cell 4A reads calibrated_materials_915mhz.json, sets relative_permittivity/conductivity/scattering_coefficient
RMSE	22.62 → 21.94 dB (+0.67 dB). Ceiling limited by geometry approx. (OSM).
Portable	Limited — values tuned for Nottingham/915 MHz

2.3 Cell 16 — MaterialMLP (Scene-Conditioned)

Property	Value
Type	Neural network that predicts ϵ_r , σ , S from 8 scene features
Architecture	Input(n_mat one-hot) + Input(8 scene feats) → FC(64,ReLU)+BN → FC(32,ReLU)+BN → 3 heads (eps_r, log
Features (8)	freq_norm, ndsm_mean, ndsm_std, building_density, tx_height, rx_h_mean, rx_h_std, urban_density
Materials	concrete, brick, glass, wet_ground, vegetation, water (6 materials)
Output	material_mlp_weights.npz + material_mlp_materials.json
Sionna 2	Load npz weights → compute 8 scene features → call predict_materials() → set $\epsilon_r/\sigma/S$
Portable	Yes — designed for any city/frequency
Note	Requires re-run of Cell 16 with dummy forward pass fix (trainable params was 0 before fix)

2.4 Cell 15 — Physics-Informed Residual MLP (Thrane et al.)

Property	Value
Type	Post-simulation RSSI correction (NOT a scene material)
Architecture	Input(45) → Dense(128,ReLU)+Dropout(0.2) → Dense(128,ReLU)+Dropout(0.2) + skip → Dense(64,ReLU) → Dense(32,ReLU) + skip → Dense(16,ReLU) + skip → Dense(8,ReLU) + skip → Dense(4,ReLU) + skip → Dense(2,ReLU) + skip → Dense(1,ReLU)
45 Features	9 groups x5: G1 Power, G2 Delay, G3/G4 Angular RX/TX, G5 Path types, G6 Vertex geometry, G7 Reference
Input	Pre-traced paths from _traced_list (Sionna 0.19 8-tuple format), rssi_sim_cached from Cell 10b
Output	residual_mlp_915mhz.npz (weights + feat_mean + feat_std)
RMSE	RT only 9.74 dB → RT + MLP 4.83 dB ($\Delta = +4.92$ dB, 50.5% improvement) N=15 test
Sionna 2	PARTIAL — MLP weights portable BUT feature extraction uses Sionna 0.19 8-tuple paths API. Feature extra
Recommended	Add post-processing cell: (1) load .npz, (2) extract 45 features from Sionna 2 paths, (3) apply $RSSI_{final} = RSSI_{sim} + residual$

2.5 Cell 14 — CNN + MLP Residual Corrector (nDSM)

Section 3 — Results Summary

Model	Notebook	RMSE Before	RMSE After	Improvement	Portable
Scalar Offset (Cell 10b)	sionna019	~22 dB	7.90 dB	~14 dB	No
Material Calib (Cell 11b)	sionna019	22.62 dB	21.94 dB	+0.67 dB	Limited
MaterialMLP (Cell 16)	sionna019	—	—	Portable ver. of 11b	Yes
Residual MLP (Cell 15)	sionna019	9.74 dB	4.83 dB	+4.92 dB (50%)	Partial
CNN+MLP (Cell 14)	sionna019	14.82 dB	5.92 dB	+8.90 dB (60%)	Yes

Literature Comparison

Reference	Reported RMSE	Our Result
Thrane et al. 2020 IEEE TVT	4–6 dB RMSE	Cell 15: 4.83 dB — matches
NVLabs Hoydis 2023	~1–2 dB gain (synth.)	Cell 11b: 0.67 dB on real-world — matches

Section 4 — Loading into Sionna 2 DEM

4.1 Currently Wired (works today)

- scalar_offset_915mhz.json → Cell 4A of sionna2 DEM (working)
- calibrated_materials_915mhz.json → Cell 4A of sionna2 DEM (working)

4.2 Cell 14 CNN — Can Be Added Directly (NO Sionna Version Dependency)

The CNN model uses only nDSM patches and scalar geometry (distance, heights, azimuth). There is no Sionna version-specific code. It can be added directly to sionna2 DEM as a post-processing cell.

```
# Add this block AFTER Sionna 2 DEM compute_fields()
import tensorflow as tf, numpy as np, rasterio
from pyproj import Transformer

cnn = tf.keras.models.load_model('results/diff_rt/cnn_residual_915mhz.h5')
ndsm = rasterio.open('ndsm.tif')

for rx_name, rssi_sim in rssi_sim_dict.items():
    rx_pos = scene.get(rx_name).position.numpy()
    patch = get_ndsm_patch(rx_pos[0], rx_pos[1])
    scalar = [dist_km/10, tx_h/100, rx_pos[2]/50, sin_az, cos_az]
    delta = float(cnn.predict([patch[np.newaxis], np.array([scalar])]))
    rssi_final[rx_name] = rssi_sim + delta
```

4.3 Cell 15 MLP — Requires Sionna 2 Path Feature Adaptation

- MLP weights ARE portable (saved as numpy arrays in .npz)
- Feature extraction `_extract_features()` uses Sionna 0.19 8-tuple paths API
- In Sionna 2, paths object has different structure (use `paths.a`, `paths.tau` directly)
- Adaptation needed: rewrite `_extract_features()` for Sionna 2 paths before using in sionna2 DEM
- This is ~50 lines of code change in the feature extraction function

4.4 Cell 16 MaterialMLP — Portable, to be Wired into Cell 4A

```
w = np.load('results/diff_rt/material_mlp_weights.npz', allow_pickle=True)
mlp = MaterialMLP(n_mat=6)
mlp(tf.eye(6), tf.zeros([1,8])) # build
mlp.set_weights([w[f'w{i}'] for i in range(len(w.files)-2)])
scene_feats = compute_scene_features(freq_hz=FREQUENCY_HZ, ndsm_data=ndsm_arr, ...)
mats = mlp.predict_materials(scene_feats)
for name, props in mats.items():
    scene.get(name).relative_permittivity = props['er']
```

Section 5 — Recommendations

#	Recommendation	Priority
1	Re-run Cell 16 (fix pushed to branch claude/cool-cori-rrWbY) — dummy forward pass added so trainable parameters are correctly in	High
2	Add Cell 14 CNN post-processing to sionna2 DEM — NO code adaptation needed, fully portable.	High
3	Adapt Cell 15 feature extraction for Sionna 2 API (~50 lines), then add to sionna2 DEM.	Medium
4	For new cities: use Cell 16 MaterialMLP — scene-conditioned, portable to any frequency/city.	Medium
5	For Nottingham only: Cell 11b JSON gives best material calibration (scene-specific tuning).	Low

End of document. Generated automatically from pipeline analysis.